Mobile Hybrid Recommender System - Complete Project Plan

Project Overview

Building a hybrid recommender system for mobile phones in India with collaborative filtering, content-based filtering, explainability, serendipity, contrarian recommendations, and bias detection.

Tech Stack & Alternatives

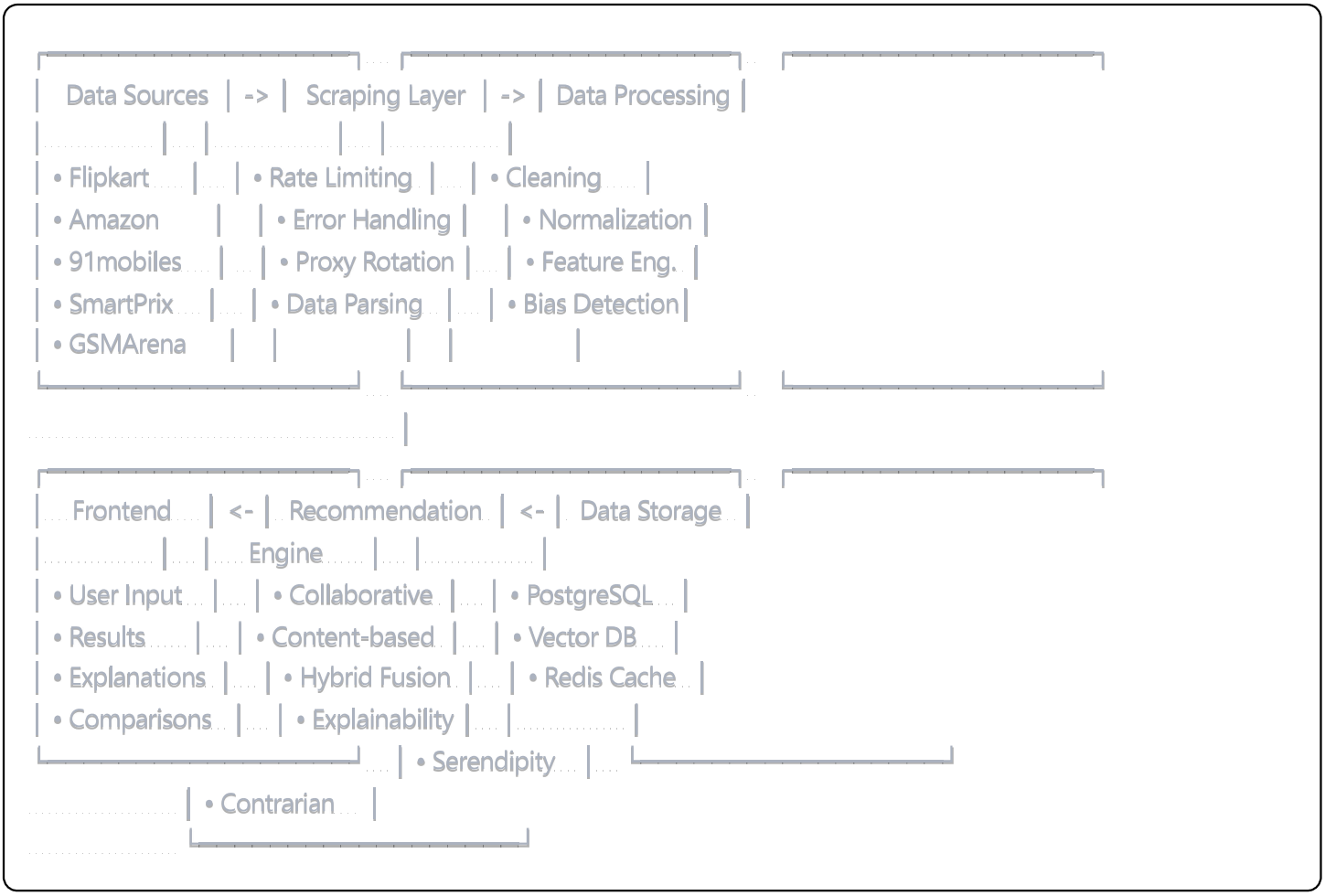
Primary Tech Stack

Component	Primary Choice	Alternatives	Reason
Backend Framework	FastAPI	Flask, Django	Async support, automatic docs
Web Scraping	Scrapy + BeautifulSoup	Selenium, Playwright	Efficient, handles rate limiting
Data Storage	PostgreSQL + Redis	MongoDB, SQLite	Structured data + caching
Vector Database	Chroma DB	Pinecone, Weaviate, FAISS	Free, local deployment
ML Framework	Scikit-learn + Pandas	TensorFlow, PyTorch	Sufficient for recommendation algorithms
LLM/NLP	Ollama (local)	Hugging Face, OpenAI API	Local deployment, cost-effective
Frontend	Streamlit	React, Flask templates	Rapid prototyping
Scheduler	APScheduler	Celery + Redis	Lightweight for this use case
Monitoring	Prometheus + Grafana	Custom logging	Industry standard

API Access Required

Service	Access Method	Purpose	Estimated Cost
RapidAPI GSMArena Parser	API Key ($9.99/month)	Phone specifications	$10/month
ScrapingBee/Scrapfly	API Key ($29/month)	Anti-bot bypass	$30/month
NewsAPI	Free tier → $449/month	Tech news	Free initially
Alternative: Custom scraping	Proxy service ($20/month)	All sources	$20/month

Detailed Architecture



Database Schema Design

Core Tables

sql

```sql
-- Phones table
CREATE TABLE phones (
    id SERIAL PRIMARY KEY,
    brand VARCHAR(50) NOT NULL,
    model VARCHAR(100) NOT NULL,
    launch_date DATE,
    price_current INTEGER,
    price_launch INTEGER,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(brand, model)
);

-- Specifications table
CREATE TABLE specifications (
    phone_id INTEGER REFERENCES phones(id),
    display_size DECIMAL(3,1),
    display_type VARCHAR(50),
    resolution VARCHAR(20),
    processor VARCHAR(100),
    ram_gb INTEGER,
    storage_gb INTEGER,
    main_camera_mp INTEGER,
    front_camera_mp INTEGER,
    battery_mah INTEGER,
    weight_g INTEGER,
    os VARCHAR(50),
    connectivity_5g BOOLEAN DEFAULT FALSE
);

-- Ratings table
CREATE TABLE ratings (
    id SERIAL PRIMARY KEY,
    phone_id INTEGER REFERENCES phones(id),
    source VARCHAR(50) NOT NULL,
    rating DECIMAL(2,1),
    review_count INTEGER,
    scraped_at TIMESTAMP DEFAULT NOW()
);

-- Derived features table
CREATE TABLE derived_features (
    phone_id INTEGER REFERENCES phones(id),
```

```
    value_for_money_score DECIMAL(3,2),
    battery_weight_ratio DECIMAL(4,2),
    camera_score DECIMAL(3,2),
    performance_score DECIMAL(3,2),
    build_quality_score DECIMAL(3,2),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- User interactions (for collaborative filtering)
CREATE TABLE user_interactions (
    id SERIAL PRIMARY KEY,
    user_id VARCHAR(100),
    phone_id INTEGER REFERENCES phones(id),
    interaction_type VARCHAR(20), -- 'view', 'like', 'compare', 'purchase'
    rating INTEGER CHECK (rating BETWEEN 1 AND 5),
    timestamp TIMESTAMP DEFAULT NOW()
);
```

Layer-by-Layer Implementation

1. Data Sourcing Layer

Web Scrapers Configuration

python

```python
# Primary sources priority
SCRAPING_SOURCES = {
    'flipkart': {
        'priority': 1,
        'rate_limit': 2,  # seconds between requests
        'base_url': 'https://www.flipkart.com',
        'selectors': {
            'price': '._30jeq3',
            'rating': '._3LWZlK',
            'specs': '._21Ahn-'
        }
    },
    'amazon': {
        'priority': 2,
        'rate_limit': 3,
        'base_url': 'https://www.amazon.in',
        'requires_proxy': True
    },
    '91mobiles': {
        'priority': 1,
        'rate_limit': 2,
        'base_url': 'https://www.91mobiles.com',
        'api_available': False
    },
    'smartprix': {
        'priority': 1,
        'rate_limit': 1,
        'base_url': 'https://www.smartprix.com'
    },
    'gsmarena': {
        'priority': 3,
        'rate_limit': 2,
        'api_endpoint': 'https://rapidapi.com/controller2042000/api/gsmarenaparser',
        'requires_api_key': True
    }
}
```

Derived Features Algorithms

```python
```

```python
def calculate_value_for_money(price, specs):
    """
    VFM = (Performance_Score + Camera_Score + Battery_Score + Build_Score) / Price_Normalized
    Scale: 0-10
    """
    performance_score = calculate_performance_score(specs)
    camera_score = calculate_camera_score(specs)
    battery_score = specs['battery_mah'] / 1000  # Normalize to 0-6 range
    build_score = calculate_build_quality(specs)

    total_score = (performance_score + camera_score + battery_score + build_score) / 4
    price_normalized = min(price / 10000, 10)  # Cap at 100k for normalization

    vfm_score = (total_score * 10) / price_normalized
    return min(vfm_score, 10)

def calculate_battery_weight_ratio(battery_mah, weight_g):
    """
    Battery efficiency considering device weight
    Higher is better
    """
    if weight_g == 0:
        return 0
    return round((battery_mah / weight_g) * 10, 2)

def calculate_performance_score(specs):
    """
    Based on processor, RAM, storage
    Scale: 0-10
    """
    processor_scores = {
        'snapdragon 8 gen 3': 10,
        'snapdragon 8 gen 2': 9.5,
        'mediatek dimensity 9200': 9.2,
        'snapdragon 888': 8.5,
        # ... more processors
    }

    processor_score = processor_scores.get(specs['processor'].lower(), 5)
    ram_score = min(specs['ram_gb'] / 2, 10)  # 20GB RAM = 10 points
    storage_score = min(specs['storage_gb'] / 25.6, 10)  # 256GB = 10 points
```

```
    return (processor_score * 0.5 + ram_score * 0.3 + storage_score * 0.2)
```

2. Scheduler Layer

python

```python
from apscheduler.schedulers.asyncio import AsyncIOScheduler
import asyncio

class DataUpdateScheduler:
    def __init__(self):
        self.scheduler = AsyncIOScheduler()

    def setup_jobs(self):
        # Daily price updates
        self.scheduler.add_job(
            self.update_prices,
            'cron',
            hour=6,
            minute=0,
            id='price_update'
        )

        # Weekly specifications check for new models
        self.scheduler.add_job(
            self.update_specifications,
            'cron',
            day_of_week='mon',
            hour=2,
            minute=0,
            id='specs_update'
        )

        # Monthly comprehensive data refresh
        self.scheduler.add_job(
            self.full_data_refresh,
            'cron',
            day=1,
            hour=1,
            minute=0,
            id='full_refresh'
        )

        # Real-time bias detection
        self.scheduler.add_job(
            self.run_bias_detection,
            'interval',
            hours=6,
```

```
            id='bias_check'
        )
```

3. Data Storage Layer

Vector Database Setup

python

```python
import chromadb
from sentence_transformers import SentenceTransformer

class VectorStoreManager:
    def __init__(self):
        self.client = chromadb.Client()
        self.collection = self.client.create_collection(
            name="phone_embeddings",
            metadata={"description": "Phone specifications embeddings"}
        )
        self.model = SentenceTransformer('all-MiniLM-L6-v2')

    def create_phone_embedding(self, phone_data):
        """Create embeddings for semantic search"""
        text_representation = f"""
        {phone_data['brand']} {phone_data['model']}
        {phone_data['processor']} {phone_data['ram_gb']}GB RAM
        {phone_data['storage_gb']}GB storage {phone_data['main_camera_mp']}MP camera
        {phone_data['battery_mah']}mAh battery {phone_data['display_size']} inch display
        """

        embedding = self.model.encode([text_representation])
        return embedding[0].tolist()
```

4. Recommendation Engine Architecture

python

```python
class HybridRecommender:
    def __init__(self):
        self.collaborative_filter = CollaborativeFilter()
        self.content_filter = ContentBasedFilter()
        self.explainer = RecommendationExplainer()
        self.serendipity_engine = SerendipityEngine()
        self.contrarian_engine = ContrarianEngine()
        self.bias_detector = BiasDetector()

    def get_recommendations(self, user_preferences, user_id=None, top_k=10):
        # Get recommendations from different engines
        collab_recs = self.collaborative_filter.recommend(user_id, top_k*2)
        content_recs = self.content_filter.recommend(user_preferences, top_k*2)

        # Hybrid fusion using weighted average
        hybrid_scores = self.fuse_recommendations(collab_recs, content_recs)

        # Apply serendipity and contrarian filters
        diverse_recs = self.add_diversity(hybrid_scores, user_preferences)

        # Bias detection and correction
        final_recs = self.bias_detector.correct_recommendations(diverse_recs)

        # Generate explanations
        explained_recs = []
        for phone_id, score in final_recs[:top_k]:
            explanation = self.explainer.explain_recommendation(
                phone_id, user_preferences, score
            )
            explained_recs.append({
                'phone_id': phone_id,
                'score': score,
                'explanation': explanation
            })

        return explained_recs
```

5. Explainability Module

python

```python
class RecommendationExplainer:
    def __init__(self):
        self.llm = self.setup_local_llm()  # Using Ollama

    def explain_recommendation(self, phone_id, user_prefs, score):
        phone_data = self.get_phone_data(phone_id)

        # Feature importance calculation
        feature_weights = self.calculate_feature_importance(phone_data, user_prefs)

        # Generate human-readable explanation
        prompt = f"""
        Explain why {phone_data['brand']} {phone_data['model']}
        is recommended for a user with preferences: {user_prefs}

        Key matching features: {feature_weights}
        Confidence score: {score}

        Provide a concise, friendly explanation in 2-3 sentences.
        """

        explanation = self.llm.generate(prompt, max_length=150)
        return explanation

    def generate_comparison_explanation(self, phone1_id, phone2_id):
        """Why phone1 is better than phone2 for this user"""
        # Implementation for comparative explanations
        pass
```

6. Serendipity & Contrarian Engines

python

```python
class SerendipityEngine:
    """Discover hidden gems and niche brands"""

    def find_serendipitous_phones(self, user_prefs, exclude_brands=None):
        # Find phones from less popular brands with excellent specific features
        niche_criteria = {
            'camera_enthusiast': 'main_camera_mp > 100 AND brand NOT IN (samsung, apple, xiaomi)',
            'battery_life': 'battery_mah > 5000 AND battery_weight_ratio > 25',
            'performance': 'performance_score > 8.5 AND price_current < 30000'
        }

        return self.query_niche_phones(niche_criteria, user_prefs)

class ContrarianEngine:
    """Recommend against popular trends with unique value"""

    def find_contrarian_recommendations(self, trending_phones, user_prefs):
        # Find phones that go against current trends but offer unique value
        contrarian_factors = [
            'compact_size',  # When everyone wants big phones
            'physical_keyboard',  # When everyone uses touch
            'removable_battery',  # When everyone seals batteries
            'headphone_jack',  # When everyone removes it
            'expandable_storage'  # When everyone goes cloud
        ]

        return self.query_contrarian_phones(contrarian_factors, user_prefs)
```

7. Bias Detection System

python

```python
class BiasDetector:
    def __init__(self):
        self.brand_bias_threshold = 0.4  # 40% of recommendations from one brand = bias
        self.price_bias_threshold = 0.3   # 30% in same price range = bias

    def detect_biases(self, recommendations):
        biases = {}

        # Brand bias detection
        brand_distribution = self.analyze_brand_distribution(recommendations)
        if max(brand_distribution.values()) > self.brand_bias_threshold:
            biases['brand_bias'] = True

        # Price range bias
        price_distribution = self.analyze_price_distribution(recommendations)
        if self.calculate_price_concentration(price_distribution) > self.price_bias_threshold:
            biases['price_bias'] = True

        # Specification bias (e.g., all high-end or all budget)
        spec_bias = self.detect_specification_bias(recommendations)
        biases['spec_bias'] = spec_bias

        return biases

    def correct_recommendations(self, recommendations):
        biases = self.detect_biases(recommendations)

        if biases:
            # Apply diversity constraints
            corrected_recs = self.apply_diversity_constraints(
                recommendations, biases
            )
            return corrected_recs

        return recommendations
```

Implementation Timeline

Phase 1: Foundation (Week 1-2)

- [ ] Set up development environment
- [ ] Create database schema
- [ ] Implement basic web scraping for 2-3 sources

- [ ] Create data processing pipeline
- [ ] Set up basic FastAPI backend

Phase 2: Data Collection (Week 3-4)

- [ ] Implement all scraping modules
- [ ] Create scheduler system
- [ ] Add error handling and retry mechanisms
- [ ] Implement data validation and cleaning
- [ ] Set up monitoring and logging

Phase 3: Recommendation Engine (Week 5-6)

- [ ] Implement collaborative filtering
- [ ] Build content-based filtering
- [ ] Create hybrid fusion logic
- [ ] Add vector database integration
- [ ] Implement basic explainability

Phase 4: Advanced Features (Week 7-8)

- [ ] Add serendipity engine
- [ ] Implement contrarian recommendations
- [ ] Build bias detection system
- [ ] Enhance explainability with LLM
- [ ] Create comparison features

Phase 5: Frontend & Polish (Week 9-10)

- [ ] Build Streamlit interface
- [ ] Add user authentication
- [ ] Implement recommendation display
- [ ] Add comparison tools
- [ ] Performance optimization
- [ ] Testing and debugging

File Structure

```
mobile_recommender/
├── src/
│   ├── scrapers/
│   │   ├── __init__.py
```

```
│   │   ├── base_scraper.py
│   │   ├── flipkart_scraper.py
│   │   ├── amazon_scraper.py
│   │   ├── mobile91_scraper.py
│   │   └── gsmarena_scraper.py
│   ├── data/
│   │   ├── __init__.py
│   │   ├── database.py
│   │   ├── models.py
│   │   └── vector_store.py
│   ├── recommendation/
│   │   ├── __init__.py
│   │   ├── collaborative.py
│   │   ├── content_based.py
│   │   ├── hybrid.py
│   │   ├── explainer.py
│   │   ├── serendipity.py
│   │   ├── contrarian.py
│   │   └── bias_detector.py
│   ├── api/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   ├── routes.py
│   │   └── schemas.py
│   ├── frontend/
│   │   ├── __init__.py
│   │   ├── app.py
│   │   ├── components/
│   │   └── utils.py
│   └── utils/
│       ├── __init__.py
│       ├── config.py
│       ├── scheduler.py
│       └── monitoring.py
├── data/
│   ├── raw/
│   ├── processed/
│   └── exports/
├── tests/
├── docker/
├── docs/
├── requirements.txt
```

```
├── docker-compose.yml
└── README.md
```

Next Steps

1. **Immediate Action Required:**
   - Sign up for RapidAPI GSMArena Parser or set up custom scraping infrastructure
   - Choose between paid scraping services vs. custom proxy setup
   - Set up development environment
2. **Key Decisions Needed:**
   - Budget for API services vs. custom scraping
   - Local deployment vs. cloud deployment
   - LLM choice: Ollama (free, local) vs. API services (paid, better quality)
3. **Risk Mitigation:**
   - Have backup scraping methods for each source
   - Implement graceful degradation for API failures
   - Regular data validation and monitoring

Would you like me to start implementing any specific component, or would you prefer to make decisions about the API access and budget first?